PHP end-of-term exam RETAKE (2020-21-1)

Due No Due Date **Points** None

Important information

- You have **120 minutes** to complete the exam. We provide you **additional 30 minutes** for filling in the `README.md` (see template below), reading the problems, downloading the materials, compressing and uploading your solution.
- Solutions must be uploaded to ELTE Canvas. **The system closes at 18:30 (Hungarian time) and there will be no opportunity to hand in your solution after the aforementioned timepoint.**
- You may use **any kind of material** (including documentation, lectures, class exercises, cheat sheets, etc) during the exam, **except human help** (synchronous, asynchronous, chat, forums, etc)!
- The problems are independent and can be solved **in any order**.
- To begin your work you should first **download the starting package we have prepared**. Each tasks has its own directory in the package. All tasks are somewhat prepared. Other than the provided files, you're free to use any other files (to store data, organize your code, implement AJAX/fetch), but make sure your solutions are in the correct folder!

```
task 1: 1/index.php
task 2: 2/index.php
task 3: 3/index.php
task 4: 4/index.php
```

- The starting package contains a template called `README.md` where you should fill in your name and Neptun code (the <> characters must be removed). **Submissions without a properly filled `README.md` file will not be graded!**

```
<Name>
<Neptun ID>
This solution was submitted and prepared by the student stated above for
the assignment of the Web programming course. I declare that this solution is my own work.
I have not copied or used third party solutions.
I have not passed my solution to my classmates, neither made it public.
Students' regulation of Eötvös Loránd University (ELTE Regulations Vol. II. 74/C. § ) states
that as long as a student presents another student's work - or at least the significant part of it -
as his/her own performance, it will count as a disciplinary fault.
The most serious consequence of a disciplinary fault can be dismissal of the student from the Univer
sity.
```

- Each directory contains a file named `TASKS.md`. Replace the space in `[ ]` with an `x` where you have (even partially) succeeded to complete the task. This will make us easier to find your solution when we grade your exam.

Task 1: End of semester duties (8 points)

At the end of semester, instructors of the *Web programming* subject still had a lot of test and home assignments to correct. Help them summarize the remaining tasks! Before coding, it is recommended to study and understand the data structure found in `index.php`.

- a. (2 points) Based on the data found in the starting `index.php` file, generate a table according to the following description:
  - the top left cell is empty.
  - the following headers for each column contains the types of tasks (eg. `exam` and `assignemnt`.) Make sure this also works if new task types are added to the data structure! Hint: Look up the `array_column` and `array_unique` functions.
- b. (2 points) Each row in the first column should contain the name of a teacher.
- c. (1 point) The name of the teachers is in alphabetical order. Hint: look up the `asort` function!
- d. (2 points) In the remaining cells, display a value in the `N / M` format. `N` is the number of **ungraded** tasks corresponding to the task type and teacher name in a given row and column. `M` is the total number of tasks, corresponding to the task type and teacher name in the row and column. Ungraded tasks are represented with their scores set to `null` in the data structure. Hint: try to create a function which calculates the numbers for any given teacher and exam, and call in it every cell!
- e. (1 point) Display each cell's content visually! Place a line break after each `N/M` value, then create a `div` with class `scale`, and set its width (in percentage) according to the ratio between `N` and `M`! (So for the imaginary result of 6 out of 9 tasks, the div should be 30% wide).

Ungraded submissions

	exam	assignment
Heu Réka	4 / 6	6 / 9
Kukor Ica	9 / 10	7 / 10
Para Zita	5 / 10	6 / 14
Programo Zoltán	8 / 9	5 / 9
Remek Elek	10 / 16	6 / 10
Riz Ottó	5 / 11	5 / 6
Teásk Anna	4 / 7	3 / 7
Trap Pista	8 / 11	8 / 11
Wincs Eszter	8 / 9	8 / 9
Zsíros B. Ödön	8 / 13	12 / 13

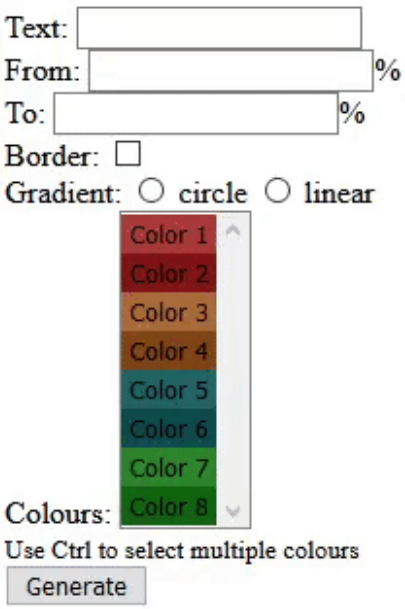
Task 2: Student feedback (8 points)

The *Web programming* teachers want to give some feedback in a well-formatted way. To do this, they use a form, where it's possible to submit the text for the feedback, the right and left side position of the textbox to be generated, a checkbox to enable a border, the type of the background gradient, and the colors of the gradient. The form and the logic used to generate the result is already done. Use PHP to validate the data of the form and produce them in the necessary format!

- `text`: required, can't be empty
- `from`: required, a number, between 0 and 100
- `to`: required, a number, between 0 and 100
- `from`, `to`: `from < to`
- `border`: not required, it's either in the `$_GET` or not
- `gradient`: required, only valid values are `circle` or `linear`
- `colours`: required, at least two color codes. You don't have to validate if the code asre actually valid hex codes.

Tasks:

- a. (1 point) validating and producing `text`
- b. (1 point) validating and producing `from` and `to`
- c. (1 point) validating `from < to`
- d. (1 point) producing `border`
- e. (1 point) validating and producing `gradient`
- f. (1 point) validating and producing `colours`
- g. (1 point) Errors are presented as a list (eg.`text` was empty, `to` is not a number, etc.)
- h. (1 point) If all input is valid, the output is displayed. Otherwise, or if $_GET is empty, nothing is displayed. (Obviously, error messages can be displayed).



Task 3: Teachers' lessons (8 points)

At the start of each semester, the *Web programming* teachers need to be assigned to practice lessons. Create an app which helps with scheduling by displaying which teacher is added to which lesson and how many lessons can a teacher do. It should also be possible to modify these data on another page, and to filter thelist of teachers based on some criteria.

- a. (1 point) List the contents of the `teachers.json` in a format similar to the example!
- b. (1 point) Generate a link on each teacher's name, which leads to the update page, passing the teacher's ID as a GET parameter.
- c. (2 points) On the update page, display the following data for each teacher: name, max number of lessons, currently assigned lessons.
- d. (1 point) Clicking the Save button on the update page stores the data and redirects to the index page. You **DO NOT** have to check the validity of the data!

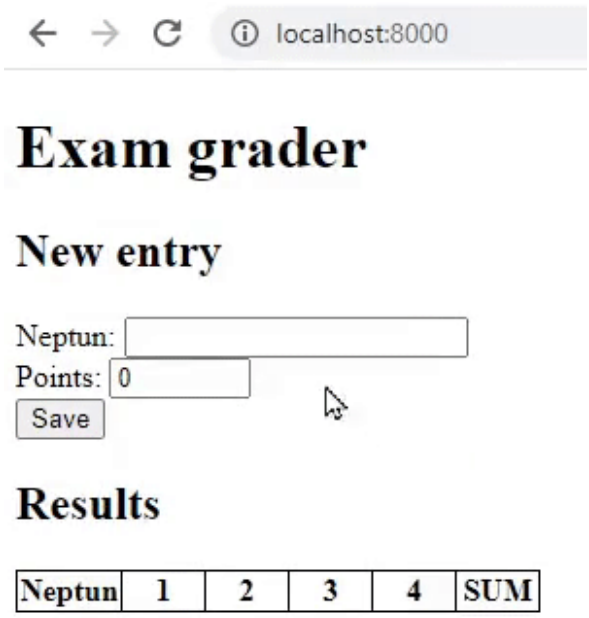
- e. (1 point) Cllicking the "Free capacity" link should list only the teachers whose max numberr of lessons is greater than their currently assigned lessons.
- f. (1 point) The "Capacity overflow" link should list only those theachers whose max number of lessons is lesser than their currently assigned lessons.
- g. (1 point) The "All teachers" link should list all teachers.



Task 4: Grading sheet (8 points)

Some teachers grade tests person by person, others correct the same task for everybody and then the next task for everybody. Create an app which helps in both ways of grading! Store the known scores in **session**, handle the data processing **on the serverside**, and do the data exchange using **AJAX/fetch**!

- a. (2 points) When loading the page, display a table from the data stored in session! If the session is empty, it is enough to display only the table headers.
- b. (3 points) Clicking the "Save" button, if the provided Neptun ID does not exist yet in the table, add it as a new row **without reloading the page**! The submitted score should be stored for task 1. Task 2 3 and 4 should stay empty, and the total score should be equal to task 1.
- c. (2 points) Clicking the "Save" button, if the given Neptun ID already exists, and not all four tasks have a score yet, update the "earliest" task's empty score to the submitted value. (eg. if a student has `7 - - -` scores, after saving, the new value should be `7 6 - -`, but if they have `7 6 5 -`, the new value shoud be `7 6 5 4`.)
- d. (1 point) If clicking the save button with the given name already in the table AND all four scores are filled, your program should do nothing.



PHP Retake			
Criteria	**Ratings**		**Pts**
Task 1 Subtask a) a. (2 points) Based on the data found in the starting index.php file, generate a table according to the following description: the top left cell is empty. the following headers for each column contains the types of tasks (eg. exam and assignemnt.) Make sure this also works if new task types are added to the data structure! Hint: Look up the array_column and array_unique functions.	**2.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	2.0 pts
Task 1 Subtask b) b. (2 points) Each row in the first column should contain the name of a teacher.	**2.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	2.0 pts
Task 1 Subtask c) c. (1 point) The name of the teachers is in alphabetical order. Hint: look up the asort function!	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 1 Subtask d) d. (2 points) In the remaining cells, display a value in the N / M format. N is the number of ungraded tasks corresponding to the task type and teacher name in a given row and column. M is the total number of tasks, corresponding to the task type and teacher name in the row and column. Ungraded tasks are represented with their scores set to null in the data structure. Hint: try to create a function which	**2.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	2.0 pts

calculates the numbers for any given teacher and exam, and call in it every cell!			
Task 1 Subtask e) e. (1 point) Display each cell's content visually! Place a line break after each N/M value, then create a div with class scale, and set its width (in percentage) according to the ratio between N and M! (So for the imaginary result of 6 out of 9 tasks, the div should be 30% wide).	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 2 Subtask a) a. (1 point) validating and producing text	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 2 Subtask b) b. (1 point) validating and producing from and to	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 2 Subtask c) c. (1 point) validating from < to	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 2 Subtask d) d. (1 point) producing border	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 2 Subtask e) e. (1 point) validating and producing gradient	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 2 Subtask f) f. (1 point) validating and producing colours	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 2 Subtask g) g. (1 point) Errors are presented as a list (eg.text was empty, to is not a number, etc.)	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 2 Subtask h) h. (1 point) If all input is valid, the output is displayed. Otherwise, or if $_GET is empty, nothing is displayed. (Obviously, error messages can be displayed).	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts

Task 3 Subtask a) a. (1 point) List the contents of the teachers.json in a format similar to the example!	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 3 Subtask b) b. (1 point) Generate a link on each teacher's name, which leads to the update page, passing the teacher's ID as a GET parameter.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 3 Subtask c) c. (2 points) On the update page, display the following data for each teacher: name, max number of lessons, currently assigned lessons.	**2.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	2.0 pts
Task 3 Subtask d) d. (1 point) Clicking the Save button on the update page stores the data and redirects to the index page. You DO NOT have to check the validity of the data!	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 3 Subtask e) e. (1 point) Cllicking the "Free capacity" link should list only the teachers whose max numberr of lessons is greater than their currently assigned lessons.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 3 Subtask f) f. (1 point) The "Capacity overflow" link should list only those theachers whose max number of lessons is lesser than their currently assigned lessons.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 3 Subtask g) g. (1 point) The "All teachers" link should list all teachers.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
Task 4 Subtask a) a. (2 points) When loading the page, display a table from the data stored in session! If the session is empty, it is enough to display only the table headers.	**2.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	2.0 pts
Task 4 Subtask b) b. (3 points) Clicking the "Save" button, if the provided Neptun ID does not exist yet in the table, add it as a new row without reloading the page! The submitted score should be stored for task 1. Task 2 3 and 4 should stay empty, and the total score should be equal to task 1.	**3.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	3.0 pts

Task 4 Subtask c) c. (2 points) Clicking the "Save" button, if the given Neptun ID already exists, and not all four tasks have a score yet, update the "earliest" task's empty score to the submitted value. (eg. if a student has 7 - - - scores, after saving, the new value should be 7 6 - -, but if they have 7 6 5 -, the new value shoud be 7 6 5 4.)	**2.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	2.0 pts
Task 4 Subtask d) d. (1 point) If clicking the save button with the given name already in the table AND all four scores are filled, your program should do nothing.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
			Total Points: 32.0